Algorithmique I

Chapitre 3 · Alternatives

Corrigé des exercices 1 à 19 (2026-2027)

LDA du cours : `if`, `else if`, `else`, `switch`, `default`, `AND`, `OR`, `NOT`, `true`, `false`, `=` pour affecter, `==` pour comparer. Les algorithmes principaux utilisent `void`, `read` et `print`.

Exercice 1 : Traçage d'algorithmes

[1, p. 30]. Chaque algorithme est exécuté deux fois, avec les lectures `(2, 3)` puis `(4, 1)`. Le tableau ne reprend que les déclarations, lectures et affectations effectivement exécutées. Les `print` sont présentés séparément.

a. exercice1a : lignes 1 à 6

Première exécution : entrées 2 et 3. Le test `a > b` (ligne 4) est faux ; la ligne 5 n'est pas exécutée.

Ligne	a	b
2 - Déclarations	-	-
3 - `read a, b`	2	3

Affichage (ligne 6) : 2.

Deuxième exécution : entrées 4 et 1. Le test `a > b` est vrai.

Ligne	a	b
2 - Déclarations	-	-
3 - `read a, b`	4	1
5 - `a = a + 2*b`	6	1

Affichage (ligne 6) : `6`. Le `print` est extérieur au `if` : il s'exécute dans les deux cas.

b. exercice1b : lignes 1 à 8

Première exécution : entrées 2 et 3. L'instruction `read b, a` place `2` dans `b` et `3` dans `a`. Le test `a > b` est vrai.

Ligne	a	b	c
2 - Déclarations	-	-	-
3 - `read b, a`	3	2	-
5 - `c = a DIV b`	3	2	1

Affichage (ligne 8) : 1.

Deuxième exécution : entrées 4 et 1. `b=4`, `a=1`, donc `a > b` est faux.

Ligne	a	b	c
2 - Déclarations	-	-	-
3 - `read b, a`	1	4	-
7 - `c = b MOD a`	1	4	0

Affichage (ligne 8) : `0`.

c. exercice1c : lignes 1 à 12

Première exécution : entrées 2 et 3. `ok` devient `false` à la ligne 5, puis `true` à la ligne 9 (`x2 == 3`).

Ligne	x1	x2	ok
2-3 - Déclarations	-	-	-
4 - `read x1, x2`	2	3	-
5 - `ok = x1 > x2`	2	3	`false`
9 - `ok = ok OR x2 == 3`	2	3	`true`
11 - `x1 = x1*100`	200	3	`true`

Affichage (ligne 12) : `203`.

Deuxième exécution : entrées 4 et 1. `ok` est `true` à la ligne 5 ; la ligne 7 conserve cette valeur (`x1 == 4`).

Ligne	x1	x2	ok
2-3 - Déclarations	-	-	-
4 - `read x1, x2`	4	1	-
5 - `ok = x1 > x2`	4	1	`true`
7 - `ok = ok AND x1 == 4`	4	1	`true`
11 - `x1 = x1*100`	400	1	`true`

Affichage (ligne 12) : `401`.

d. exercice1d : lignes 1 à 12

Première exécution : entrées 2 et 3. `z=24` ; le `switch` choisit le cas `14, 24` (ligne 7).

Ligne	x	y	z	ok
2-3 - Déclarations	-	-	-	-
4 - `read x, y`	2	3	-	-
5 - `z = 10*x + y + 1`	2	3	24	-
7 - `ok = z MOD 6 == 0`	2	3	24	`true`
11 - `z = 2*z`	2	3	48	`true`

Affichage (ligne 12) : `48`.

Deuxième exécution : entrées 4 et 1. `z=42` ; le `switch` choisit `default` (ligne 9).

Ligne	x	y	z	ok
2-3 - Déclarations	-	-	-	-
4 - `read x, y`	4	1	-	-
5 - `z = 10*x + y + 1`	4	1	42	-
9 - `ok = z MOD 7 == 0`	4	1	42	`true`
11 - `z = 2*z`	4	1	84	`true`

Affichage (ligne 12) : `84`.

Exercice 2 : Simplification de code

[1, p. 31]. Les six fragments sont présentés dans l'ordre des colonnes gauche/droite, de haut en bas.

a. `if ok == true` :

```
if ok
    print nombre
```

b. `if ok == false` :

```
if NOT ok
    print nombre
```

c. Valeur de `cool` selon `nerveux` :

```
cool = NOT nerveux
```

d. `a > b` / `a ≤ b` :

```
ok = a ≤ b
```

e. Mettre `ok2` à faux seulement si `ok1` est faux :

```
ok2 = ok1 AND ok2
```

f. Mettre `ok3` à faux si `ok1` est vrai et `ok2` est faux ; dans les autres cas, ne pas modifier `ok3` :

```
if ok1 AND NOT ok2
    ok3 = false
```

Vigilance : en e et f, les valeurs des variables déjà définies doivent être conservées lorsque l'affectation initiale n'a pas lieu.

Exercice 3 : Majeur ou mineur ?

[1, p. 31]. Âge entier supposé non négatif.

```
void majorite()
    int age
    read age
    if age < 18
        print "mineur"
    else
        print "majeur"
```

Frontière : 17 ans → mineur ; 18 ans → majeur.

Exercice 4 : Affaire de signe

[1, p. 31].

```
void signe()
    float x
    read x
    if x < 0
        print "strictement négatif"
    else if x > 0
        print "strictement positif"
    else
        print "nul"
```

Essais : `-2`, `0`, `3.5` couvrent les trois branches.

Exercice 5 : Équation du second degré

[1, p. 31]. Une équation du second degré impose `a` $\neq$ `0`. Le code traite aussi le cas où `a=0`, par précaution.

```
void equation()
    float a, b, c, delta, x1, x2
    read a, b, c
    if a == 0
        if b ≠ 0
            print -c/b
        else if c == 0
            print "Tous les réels sont solutions"
        else
            print "Aucune solution"
    else
        delta = b*b - 4*a*c
        if delta < 0
            print "Aucune solution réelle"
        else if delta == 0
            print -b/(2*a)
        else
            x1 = (-b - sqrt(delta))/(2*a)
            x2 = (-b + sqrt(delta))/(2*a)
            print x1, x2
```

Contrôles : `(1,-3,2)` $\rightarrow$ racines 1 et 2 ; `(1,-2,1)` $\rightarrow$ racine double 1 ; `(1,0,1)` $\rightarrow$ aucune racine réelle. Ne calculer `sqrt(delta)` que pour `delta` $\geq$ `0`.

Exercice 6 : Maximum de 3 nombres

[1, p. 31].

```
void maximumTrois()
    float a, b, c, maximum
    read a, b, c
    maximum = a
    if b > maximum
        maximum = b
    if c > maximum
```

```
        maximum = c
    print maximum
```

Essai : `(-8,-2,-5)` → `-2`. Ne pas initialiser le maximum à zéro lorsque les entrées peuvent être négatives.

Exercice 7 : Les Poissons

[1, p. 32]. Date d'anniversaire supposée valide.

```
void signePoissons()
    int jour, mois
    Boolean poissons
    read jour, mois
    poissons = (mois == 2 AND jour ≥ 19) OR
               (mois == 3 AND jour ≤ 20)
    if poissons
        print "Poissons"
    else
        print "Pas Poissons"
```

Bornes : 18/02 → faux ; 19/02 → vrai ; 20/03 → vrai ; 21/03 → faux.

Exercice 8 : Jour de la semaine

[1, p. 32]. Novembre 2025 commence un samedi. Entrée `jour` entre 1 et 30.

```
void jourNovembre2025()
    int jour, reste
    read jour
    reste = jour MOD 7
    switch reste
        1 : print "samedi"
        2 : print "dimanche"
        3 : print "lundi"
        4 : print "mardi"
        5 : print "mercredi"
        6 : print "jeudi"
        0 : print "vendredi"
```

Contrôles : 1 → samedi ; 6 → jeudi ; 7 → vendredi ; 8 → samedi ; 30 → dimanche. La correspondance est propre à novembre 2025.

Exercice 9 : Nombre de jours du mois

[1, p. 32]. Année 2028, bissextile ; mois entre 1 et 12.

```
void joursDuMois2028()
    int mois, jours
    read mois
    switch mois
        1, 3, 5, 7, 8, 10, 12 : jours = 31
        4, 6, 9, 11 : jours = 30
        2 : jours = 29
    print jours
```

Contrôles : janvier → 31 ; février 2028 → 29 ; avril → 30.

Exercice 10 : Année bissextile

[1, p. 32].

```
void anneeBissextile()
    int annee
    Boolean bissextile
    read annee
    bissextile = (annee MOD 400 == 0) OR
                 ((annee MOD 4 == 0) AND (annee MOD 100 ≠ 0))
    if bissextile
        print "Année bissextile"
    else
        print "Année non bissextile"
```

Contrôles : 2022 → non ; 2024 → oui ; 2100 → non ; 2400 → oui. Année positive du calendrier grégorien.

Exercice 11 : La cote finale d'ALG1

[1, p. 32]. Chaque cote est sur 20. Le bilan compte pour 25 % seulement lorsqu'il dépasse la cote d'examen.

```
void coteFinale()
    float bilan, examen, finale
    read bilan, examen
    if bilan > examen
        finale = 0.25*bilan + 0.75*examen
    else
        finale = examen
    print finale
```

Contrôles : bilan 16, examen 12 → 13 ; bilan 12, examen 16 → 16 ; égalité 14/14 → 14.

Exercice 12 : Le stationnement alterné

[1, p. 33]. Date et numéro de maison supposés valides. Première moitié du mois : numéros impairs ; seconde moitié : numéros pairs.

```
void stationnement()
    int jour, mois, numero
    Boolean correct
    read jour, mois, numero
    correct = (jour ≤ 15 AND numero MOD 2 ≠ 0) OR
              (jour > 15 AND numero MOD 2 == 0)
    if correct
        print "Stationnement correct"
    else
        print "Stationnement incorrect"
```

Contrôles : 15 du mois, n° 7 → correct ; 16 du mois, n° 7 → incorrect ; 16 du mois, n° 8 → correct. Le mois fait partie de la date lue mais n'influence pas cette règle.

Exercice 13 : La boule de cristal

[1, p. 33]. Heure entre 0 et 23, minute entre 0 et 59.

```
void minuteSuivante()
    int heure, minute
    read heure, minute
    minute = minute + 1
    if minute == 60
        minute = 0
        heure = (heure + 1) MOD 24
    print "Dans une minute, il sera ", heure, "h ", minute
```

Exemple exigé : 21h32 → `Dans une minute, il sera 21h 33`. **Limites :** 21h59 → 22h 0 ; 23h59 → 0h 0. Le cours ne prescrit pas de zéro initial pour la minute dans cet exercice.

Exercice 14 : What time is it ?

[1, p. 33]. Conversion d'une heure 0-23 dans le système anglais sur 12 heures.

```
void heureAnglaise()
    int heure, minute, heure12
    String periode
    read heure, minute
    if heure < 12
        periode = "AM"
    else
        periode = "PM"
    heure12 = heure MOD 12
    if heure12 == 0
        heure12 = 12
    if minute < 10
        print heure12, ":0", minute, " ", periode
    else
        print heure12, ":", minute, " ", periode
```

Contrôles : 03h10 → `3:10 AM` ; 12h25 → `12:25 PM` ; 15h21 → `3:21 PM` ; 00h05 → `12:05 AM`.

Exercice 15 : Prix des photocopies

[1, p. 33]. Le tarif à l'unité est déterminé par la quantité totale. Il ne s'agit pas de tranches progressives.

```
void photocopies()
    int quantite
    float prixUnitaire, total
    read quantite
    if quantite < 10
        prixUnitaire = 0.10
    else if quantite < 100
        prixUnitaire = 0.07
    else if quantite < 1000
        prixUnitaire = 0.05
    else
        prixUnitaire = 0.04
    total = quantite*prixUnitaire
    print total
```

Seuils : 9 copies → 0,90 € ; 10 → 0,70 € ; 100 → 5 € ; 1000 → 40 €. Quantité entière ≥ 0. Les baisses du total aux seuils résultent du tarif unitaire unique.

Exercice 16 : Le triangle isocèle

[1, p. 34]. Le troisième angle vaut `180-a-b` degrés.

```
void triangleIsocele()
    float a, b, c
    read a, b
    c = 180 - a - b
    if a ≤ 0 OR b ≤ 0 OR c ≤ 0
        print "Angles invalides"
    else if a == b OR a == c OR b == c
        print "Triangle isocèle"
    else
        print "Triangle non isocèle"
```

Contrôles : 50°, 80° → troisième angle 50°, isocèle ; 60°, 60° → équilatéral et donc isocèle ; 30°, 60° → non isocèle.

Exercice 17 : Les saisons

[1, p. 34]. Respecter les dates de l'énoncé. Entrée supposée être une date valide.

```
void saison()
    int jour, mois, cle
    read jour, mois
    cle = 100*mois + jour
    if cle ≥ 321 AND cle < 621
        print "printemps"
    else if cle ≥ 621 AND cle < 921
        print "été"
    else if cle ≥ 921 AND cle < 1221
        print "automne"
    else
        print "hiver"
```

Bornes : 20/03 → hiver ; 21/03 → printemps ; 20/06 → printemps ; 21/06 → été ; 21/12 → hiver.

Exercice 18 : Les promos

[1, p. 34]. La remise porte sur le prix total pour un même produit.

```
void promotions()
    float prixUnitaire, remise, total
    int quantite
    read prixUnitaire, quantite
    if quantite < 5
        remise = 0
    else if quantite < 10
        remise = 0.05
    else if quantite < 50
        remise = 0.10
    else
```

```
        remise = 0.15
    total = quantite*prixUnitaire*(1 - remise)
    print total
```

Seuils : à 10 €/unité, 4 articles → 40 € ; 5 → 47,50 € ; 10 → 90 € ; 50 → 425 €. Prix et quantité non négatifs.

Exercice 19 : Moins fort !

[1, p. 35]. Volume entier de 0 à 20 ; commande `"p"` (plus) ou `"m"` (moins).

```
void volumeTeleviseur()
    int volume
    String bouton
    read volume, bouton
    if bouton == "p" AND volume < 20
        volume = volume + 1
    else if bouton == "m" AND volume > 0
        volume = volume - 1
    print volume
```

Contrôles : 20 et `"p"` → 20 ; 0 et `"m"` → 0 ; 10 et `"p"` → 11 ; 10 et `"m"` → 9. Les autres commandes sont hors du domaine de l'exercice.

Références

[1] ESI, HE2B, *Algorithmique I : syllabus 2026-2027*, Bruxelles, Belgique, 2026.